SMART-G Playground: user manual

A reference for every field of the playground window (`smartg_playground.py`), with what it does, its values and its defaults, plus the 3D view, the output files and a few practical tips.

1. What the playground is

The playground is an interactive sandbox for the cloud-imaging simulation. You place a satellite (or the whole formation) above a cloud field, choose where the sun is and what the camera sees, and press **Render**: SMART-G, a GPU Monte Carlo radiative-transfer code, traces photons through the 3D cloud and returns the reflectance that each pixel would record. The images are saved to disk.

Quick start

- â€¢ Start it with `python smartg_playground.py` (use the Anaconda Python that has SMART-G installed). It opens at the satellites' current positions.
- â€¢ Pick a **Sandbox mode** and, for one satellite, an off-nadir angle (row 1–2 in Figure 1). Move the sun with the two sun sliders (row 3).
- â€¢ Leave **Image resolution** at Tiny and **Photons per pixel** at 2000 for a first, fast render (row 5). Press **Render**. The result is in the status line and in the images folder.
- â€¢ Press **Voxels per pixel** to see which resolution suits your cloud, raise the resolution and photons, then **Save configuration…** (row 8) to render the same scenario on a server.

2. The window at a glance

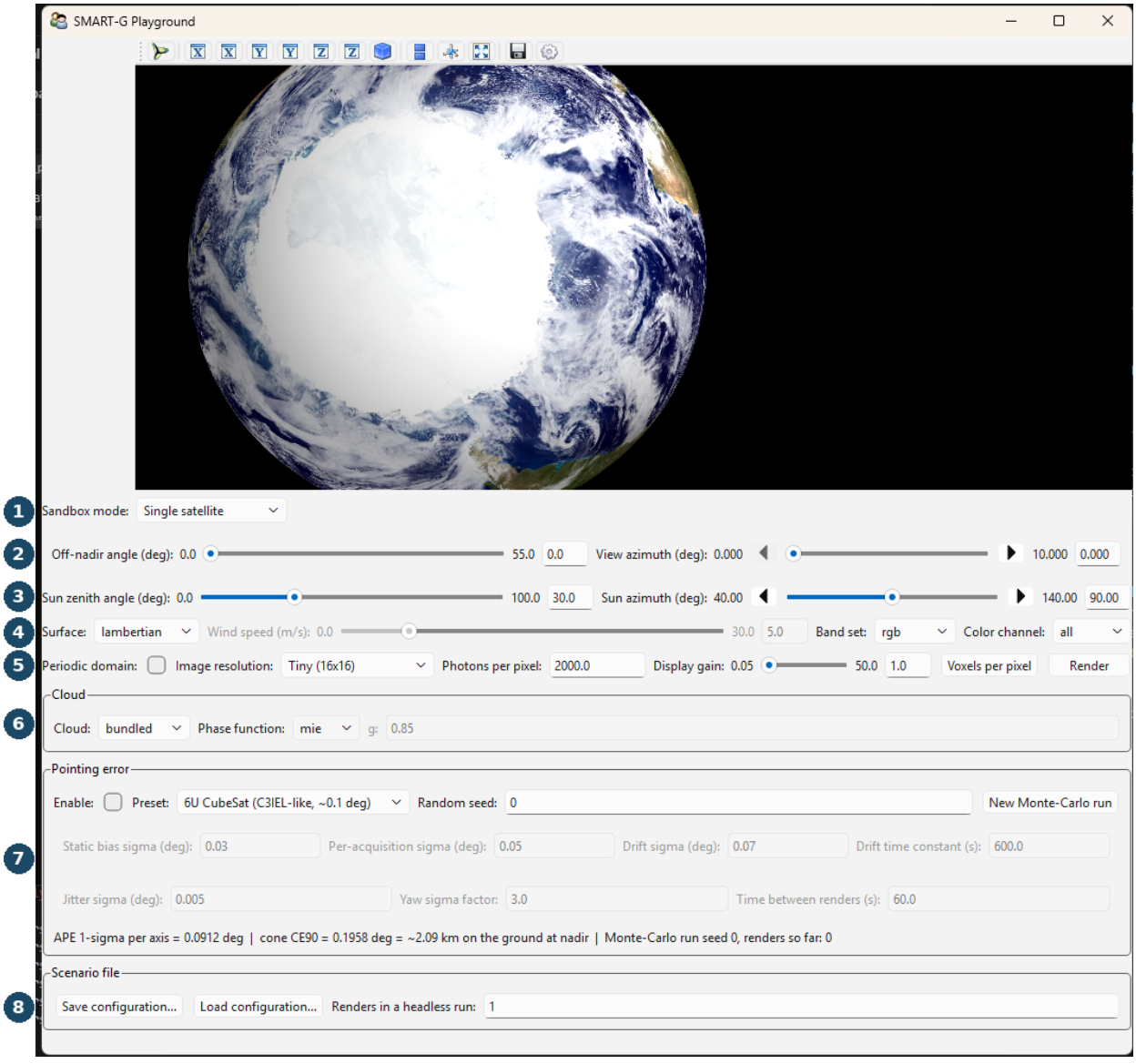


Figure 1. The window with its default settings. The numbers mark the rows described in the next sections: (1) sandbox mode, (2) viewing direction of the single satellite, (3) the sun, (4) surface and spectral settings, (5) image and render controls, (6) the cloud, (7) the pointing error, (8) the scenario file. The status line is the text area under the last box.

Some rows only appear in some states: the satellite check boxes in formation mode, the camera range for the *grid* band set, and the BOMEX fields for the *bomex* cloud (Figure 2).

3. The 3D view

The black area on top shows the Earth with the satellites. Rotate, zoom and pan it with the mouse, as in any Mayavi scene; the toolbar above it has the standard view buttons. The view is centred on the spot the satellite(s) look at, not on the Earth's centre. What you see changes with the controls:

Symbol	Meaning
Red sphere	The satellite (single mode).
Green line	Its line of sight, from the satellite down to the Earth. It moves with the off-nadir angle and the view azimuth.
Yellow sphere	**Point a**: where the line of sight meets the ground (single mode), or the shared ground target of the formation. The cloud field is centred on it.
Orange arrow	The direction of the sun at the target, set by the two sun controls.
Red spheres and cyan arrows	Formation mode: the 10 satellites and the direction each one points in.

4. The fields, row by row

Row 1: Sandbox mode

Field	What it does	Values / default
Sandbox mode	**Single satellite**: one satellite at its real position (from the orbit data it was started with), looking where you choose. **Formation (frozen)**: the CloudCT formation of 10 satellites spaced 100 km along the orbit, frozen at one moment. Satellite 4 looks at its own nadir and the other nine look at the same ground point.	Single satellite, Formation (frozen)

Row 2: viewing direction (single mode) or satellites (formation mode)

Field	What it does	Values / default
Off-nadir angle (deg)	The angle between the line of sight and straight down from the satellite. 0 is nadir. The limit keeps the ray well short of the horizon. Note it is the angle *at the satellite*; at the ground the angle from the vertical is larger because of the Earth's curvature (30° becomes about 33°), and the render reports that ground angle.	0 to 55; default 0
View azimuth (deg)	The direction in which the line of sight is tilted, measured counter-clockwise from local east (90 is north). It has no effect at 0° off-nadir.	0 to 360; default 0
Satellites to render	Formation mode: tick the satellites to render (Sat 0 to Sat 9). Each ticked satellite is a separate SMART-G run, one after the other, so the time grows with their number.	default: Sat 4 only

Row 3: the sun

Field	What it does	Values / default
Sun zenith angle (deg)	The angle of the sun from the local vertical at the target. 0 is overhead; values above 90 put the sun below the horizon.	0 to 100; default 30
Sun azimuth (deg)	The direction of the sun, measured counter-clockwise from local east. The orange arrow in the 3D view shows it. The sun is set directly, not from the date and time.	0 to 360; default 90

The sun is treated as a distant point source (a parallel beam). These are two independent angles; they do not follow the satellite's orbit.

Row 4: surface and spectral settings

Field	What it does	Values / default
Surface	**lambertian**: a flat, dark, diffusely reflecting ground (albedo 0.05). **ocean**: a wind-roughened sea, which gives sun glint and angle-dependent reflection; it reflects only (no water-leaving light, so no sea colour). It applies to the whole domain.	lambertian, ocean; default lambertian
Wind speed (m/s)	Wind over the sea, which sets how rough it is (Cox-Munk model). Greyed out unless the surface is ocean.	0 to 30; default 5
Band set	What each colour channel is. **rgb**: three single wavelengths, 440 nm (blue), 550 nm (green), 670 nm (red). **modis**: the MODIS Terra channels b03, b04 and b01 (456–474, 545–565 and 625–667 nm), simulated as REPTRAN bands (one representative wavelength each). **grid**: a grid of wavelengths over a camera's range, where each photon's wavelength is drawn in proportion to the camera response times the solar irradiance; the camera response is uniform for now.	rgb, modis, grid; default rgb
Color channel	**all**: three channels, giving a colour image. **blue**, **green** or **red**: only that channel, giving a grayscale image. The photons are shared among the simulated channels, so one channel on its own is less noisy than the same channel in a three-channel run with the same photons per pixel.	all, blue, green, red; default all

Camera range and step (only for the grid band set; row B in Figure 2)

Field	What it does	Values / default
Camera range from / to (nm)	The wavelength range the camera measures. It is split into equal thirds for blue, green and red (400–500, 500–600 and 600–700 nm by default).	300 to 4000; default 400 to 700
Grid step (nm)	The width of the wavelength bins. One wavelength is simulated per bin, at its centre; smaller steps sample the band more finely (a 10 nm step over 400–700 nm is 30 wavelengths). At most a third of the range.	0.5 to 1000; default 10

Row 5: image and render

Field	What it does	Values / default
Periodic domain	Unticked: the cloud field has a hard edge with an empty margin of 30 km around it; a photon that leaves is lost. Ticked: the field repeats at its edges, like an endless cloud deck, and the images tile.	default off
Image resolution	The image size in pixels (a preset). SMART-G uses one sensor per pixel, so run time and memory grow roughly with the pixel count. Larger presets are slow. The window starts at Tiny; a saved scenario file defaults to Small.	Tiny 16×16, Small 32×32, Medium 64×64, Large 128×128, XL 256×256, XXL 512×512, Huge 1024×1024, Low 4:3 256×192, High 4:3 1024×768
Photons per pixel	How many photons are launched for each pixel; the total is this number times the pixel count. The noise (speckle) falls as the square root of it: four times the photons halves the noise. With several channels the photons are shared among them.	1 to 1 000 000; default 2000
Display gain	The brightness of the saved PNG only: each pixel is shown as $(\text{gain} \times \text{reflectance})^{1/1.8}$, clipped at 1. Raise it to brighten dark scenes (the sea is dark), lower it to see detail in bright clouds. The raw reflectance file is not affected.	0.05 to 50; default 1.0
Voxels per pixel	A button. It does not render. It estimates how many voxels of the cloud one pixel covers at the chosen resolution, and how many voxels a line of sight shares with its neighbours', for the current view (or for each satellite ticked in formation mode). When pixels are much smaller than the voxels, neighbouring pixels see the same ones and a lower resolution loses nothing. One line per view appears in the status line; the full report is printed in the terminal.	button

Field	What it does	Values / default
Render	A button. It runs SMART-G for the current settings and saves the images (section 5). The status line shows progress and, afterwards, a summary. The window stays busy during a render.	button

Row 6: the cloud

Field	What it does	Values / default
Cloud	**bundled**: the cumulus field that comes with SMART-G (a 6.67 km square). **bomex**: a cloud built from a BOMEX droplet-spectrum netCDF file (rows C and D in Figure 2).	bundled, bomex; default bundled
Phase function	How the cloud droplets scatter light. **mie**: the phase functions of SMART-G's water-cloud table (Mie theory, chosen by each voxel's droplet size). **hg**: a Henyey-Greenstein function with the asymmetry parameter g. It applies to either cloud.	mie, hg; default mie
g	The asymmetry parameter of the Henyey-Greenstein function: 0 is isotropic, values near 1 scatter strongly forward. Water clouds are about 0.85. Greyed out unless the phase function is hg.	−0.99 to 0.99; default 0.85
BOMEX file / Browse…	bomex only. The path of the netCDF file with the droplet size distribution (the files ending `com3D_int_2.nc`, which contain FNCD). Type it, or press Browse. The first render with a file builds a cloud file from it (about half a minute for the whole field) and keeps it for later runs.	path
Extinction	bomex only. How the extinction coefficient of each voxel is computed. **q2**: the paper's approximation, extinction efficiency Q = 2. **mie**: the Mie extinction efficiency of each droplet-size bin (a few percent higher).	q2, mie; default q2
Crop (cells)	bomex only. 0 keeps the whole field (512 × 512 cells of 20 m). A number of at least 16 crops a square of that many cells around the thickest column.	0, or 16 and more; default 0

Row 7: pointing error

A model of how the real satellite's pointing differs from the commanded one. When enabled, every Render is one *acquisition*: the line of sight is turned by a random error, and the image footprint moves and rotates accordingly. All the sigmas below are 1σ per axis across the line of sight (roll and pitch); the yaw (about the line of sight) is larger by the yaw factor. The error is the sum of four parts.

Field	What it does	Values / default
Enable	Turns the pointing error on. The fields below are greyed out when it is off.	default off
Preset	Fills the five sigmas with a typical set: **6U CubeSat (C3IEL-like, ~0.1°)** (bias 0.03, per-acquisition 0.05, drift 0.07, time constant 600 s, jitter 0.005) or **Agile EO satellite (~0.01°)** (0.003, 0.005, 0.007, 1000 s, 0.0005). Editing any sigma switches it to **Custom**.	default 6U CubeSat
Random seed	The seed of the random draws. The same seed gives the same sequence of errors.	integer; default 0
New Monte-Carlo run	A button that adds one to the seed: new static biases and drifts for every satellite.	button
Static bias sigma (deg)	An error that is fixed for each satellite throughout one run (misalignment, calibration).	0 to 10; default 0.03
Per-acquisition sigma (deg)	An error drawn again at every Render (controller residual, star-tracker bias).	0 to 10; default 0.05
Drift sigma (deg) and time constant (s)	A slowly varying error that is correlated between renders, with the given standard deviation and correlation time (a first-order Gauss-Markov process).	0.07 deg; 600 s
Jitter sigma (deg)	A fast, uncorrelated error: one random sample per render.	0 to 10; default 0.005
Yaw sigma factor	The factor by which the sigmas about the line of sight are larger than those across it (typically 2 to 5).	0 to 10; default 3.0

Field	What it does	Values / default
Time between renders (s)	The time step between successive acquisitions; it sets how much the drift has moved between two renders.	default 60
Summary line	Under the boxes: the combined 1σ error per axis (root-sum-square of the four parts), the 90% line-of-sight error (CE90, 2.146 times that), what it means on the ground at nadir in km, the seed, and the number of renders so far.	information

The status line after a render gives each satellite's line-of-sight error, its ground offset (east and north, in km) and its yaw.

Row 8: scenario file

Field	What it does	Values / default
Save configuration…	Writes every setting to a JSON file (the dialog opens in the `configs` folder; default name `scenario.json`). It also stores the moment and orbit data that the satellite geometry was computed from, so a later run reproduces exactly the same geometry.	button
Load configuration…	Applies the settings of a saved file. The geometry is *not* applied from inside a running window; to restore it too, start the program with the file name: `python smartg_playground.py configs/scenario.json`. The status line says if the geometry differs.	button
Renders in a headless run	Saved with the configuration. How many acquisitions a headless run makes. It matters only with the pointing error on, since each acquisition draws fresh errors.	1 to 100 000; default 1

The status line

The text area at the bottom of the window. It shows messages (mode changes, progress such as “Building the cloud file…”), the result of a render and of Voxels per pixel, and errors (“Render failed: …”), which do not close the window. After a render it lists the ground viewing angles, the files that were saved, and per channel the minimum, median, 99th percentile and maximum of the raw reflectance, with the share of pixels that the display gain clips to white.

Rows that appear in other states (Figure 2)

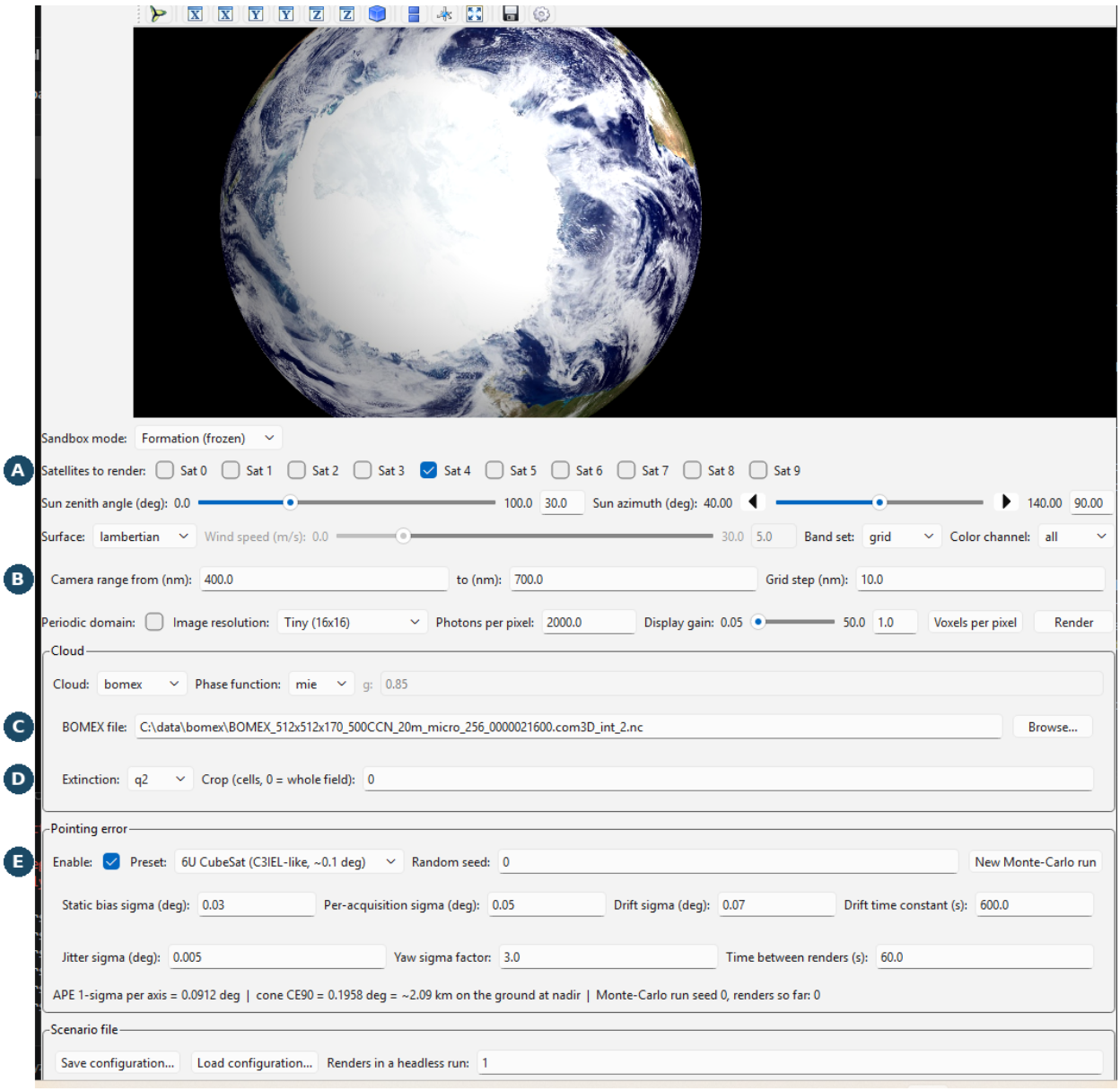


Figure 2. The window in formation mode, with the grid band set, the BOMEX cloud and the pointing error enabled. (A) The satellite check boxes replace the viewing-direction row. (B) The camera range and grid step. (C) The BOMEX file path and Browse button. (D) Extinction and crop. (E) The pointing-error fields become active.

5. What a render produces

Images are written to the folder `smartg_playground_images`, next to the program. Each render creates, for the image *name*:

File	Content
name.png	The picture you look at, with the display gain and gamma applied. One colour image for three channels, grayscale for one.
name.npy	The raw reflectance, before gain, gamma and clipping: an array of shape (height, width, number of channels), rows from top to bottom. The channels are in the order blue, green, red. Load it with `numpy.load`.
*name*_raw.png	One heat map per channel of the raw reflectance on a shared scale, with colour bars.
*name*_spectral.npz	Only for the grid band set: the reflectance at every simulated wavelength, with the wavelengths, the share of photons each got and the channel weights, so that another camera response or sun spectrum can be applied later without a new render.

Names. A single-satellite image is called, for example, `single_offnadir20_az0_sza30_saa90_nonperiodic.png`; in formation mode the images go into a folder named after the settings, one `satellite_NN.png` per satellite. Settings that differ from the original ones are added to the name, so different runs do not overwrite each other: `_ocean5ms` (the sea), `_bomex_q2_crop200` (the BOMEX cloud, its extinction and crop), `_hg0.85` (the phase function), `_modis` or

_grid (the band set), _green (one channel), and _noise_seed0_render1 (the pointing error).

6. Practical tips

- â€¢ **Start small.** Check the geometry with Tiny and 2000 photons per pixel, then raise the resolution and photons. The speckle you see at low photons is Monte Carlo noise, not a bug.
- â€¢ **Choose the resolution with Voxels per pixel.** Beyond the point where a pixel is as small as a voxel (about 512 pixels across for the BOMEX field at nadir, about 100 for the bundled cloud), more pixels add no information.
- â€¢ **Photons cost time, not memory.** The run time follows the total number of photons, so doubling the pixels while keeping the photons per pixel doubles it; the noise per pixel stays the same.
- â€¢ **Several channels share the photons.** If one channel is all you need, choosing it gives that channel the full photon count.
- â€¢ **Heavy runs belong on a server.** Save the configuration and run it without the window with run_from_config.py, or run a folder of BOMEX files with run_bomex_headless.py; estimate_voxels_headless.py does the voxels-per-pixel estimate without the window (see HEADLESS_PIPELINE.md).

7. If something goes wrong

Message or symptom	Likely cause and remedy
“Render failed: ...” mentioning a missing file or auxiliary data	SMART-G's auxiliary data (smartg_auxdata) is not where the program looks for it; the bundled cloud needs smartg_auxdata/IPRT. Set the environment variable SMARTG_DIR_AUXDATA or download the data.
“BOMEX dataset not found”	The path in the BOMEX file field is wrong or the file is on another machine. Use Browse.
The window seems frozen after Render	A render is running (a large resolution or many photons can take many minutes). The window is busy until it finishes.
The first BOMEX render takes long	The cloud file is being built from the netCDF (about half a minute for the whole field); later renders reuse it.
Out-of-memory error with the grid band set	Too many wavelengths for the GPU or the host memory; make the grid step larger or the range narrower.
The window is taller than the screen	Enlarge it by maximising or dragging its edge; the status line is at the bottom.